Architecting the AI-Augmented Oral History Pipeline: A Comprehensive Technical Analysis for Automated Narrative Generation

1. Executive Summary and Architectural Vision

The convergence of generative artificial intelligence, high-fidelity speech recognition, and large-context linguistic modeling has created a novel paradigm for literary creation: the Automated Oral History Pipeline. This report provides an exhaustive technical analysis and architectural blueprint for a system designed to transform unstructured voice recordings into a coherent, stylized manuscript. The proposed solution addresses the specific requirements of ingesting casual voice narration, extracting discrete narrative events, maintaining a persistent and evolving "Context File" (or Story Bible), and generating prose that strictly adheres to the narrator's unique authorial voice.

Unlike traditional dictation software, which functions as a direct speech-to-text typewriter, the system architecture proposed herein acts as a cognitive intermediary. It functions not merely as a scribe, but as a structural editor and archivist. The core challenge this report addresses is the translation of non-linear, oral storytelling—often characterized by fragmentation, digression, and temporal jumping—into a linear, structured written format. To achieve this, we move beyond simple transcription to implement a multi-layered data processing pipeline involving semantic analysis, entity tracking, and stylometric adaptation.

The recommended technology stack leverages a hybrid approach: **local processing** for sensitive and high-bandwidth audio ingestion, and **cloud-based reasoning** for complex narrative construction. Specifically, we identify the integration of **OpenAI's Whisper Large V3** (for transcription accuracy), **Google's Gemini 1.5 Pro** (for massive context management via caching), and **Anthropic's Claude 3.5 Sonnet** (for superior stylistic prose generation) as the optimal triad of models for 2025-2026. This is orchestrated via a Python-based logic layer that handles file synchronization, event extraction, and automated compilation using **Pandoc**.

This document details the selection logic for each component, backed by comparative analysis of current benchmarks, cost structures, and technical capabilities, providing a roadmap for building a system that allows an author to simply speak and have a book materialize in the background.

---

2. The Theoretical Framework: From Orality to Digital Text

To architect a robust technical solution, one must first understand the fundamental transformation required: converting "Orality" to "Literacy." Oral storytelling differs efficiently from written text. It relies on intonation, repetition, and shared context, whereas written text requires explicit structure, varied vocabulary, and clear antecedents.

2.1 The Cognitive Load of Dictation

When a user narrates a story, they rarely speak in finished prose. They speak in "thought units" rather than grammatical sentences. A raw transcript of a normal voice recording is often unreadable as a book chapter; it is filled with disfluencies (um, ah), false starts ("So I went to the—well, actually, before that..."), and implicit context.

A standard "Speech-to-Text" (STT) tool preserves these errors. For this project, the requirement to "format the text how I write" implies a transformation layer that cleans the "signal noise" of speech while preserving the "voice" of the author. This requires a two-step generative process: first, a faithful transcription to capture the data, and second, a "transmutation" pass where an Large Language Model (LLM) rewrites the content.

2.2 The Non-Linearity of Memory

The user's requirement to "arrange files automatically according to event" highlights a critical architectural challenge: **Temporal Decoupling**. The time a recording is made (Recording Time) rarely matches the time the event occurred in the story (Narrative Time). An author might record the ending of the book on Day 1 and the beginning on Day 20.

Therefore, the system cannot rely on file system timestamps (like 2026-01-26_Recording.mp3) for organizing the book. Instead, it must perform **Semantic Event Extraction**. The system must "listen" to the content, identify temporal markers (e.g., "Back when I was ten..."), and map that segment to a specific slot in the narrative timeline. This necessitates a sophisticated "Context File" that acts as a database of the story's timeline, updated dynamically with every new recording.

---

3. The Ingestion Layer: Audio Acquisition and Synchronization

The first technical hurdle is the "Input Gap"—the friction involved in transferring a voice note from a mobile device to the processing pipeline. If the user must manually export, email, or upload files, the workflow breaks. The solution requires an automated, background

synchronization protocol that treats the cloud storage as a "Watch Folder."

3.1 Mobile Capture Ecosystems and Protocols

The choice of mobile ecosystem determines the automation capabilities. We analyze the two dominant platforms, Android and iOS, to recommend the optimal ingestion workflow.

3.1.1 Android Architecture: The Filesystem Approach

Android's open filesystem architecture allows for robust, two-way background synchronization. This is the preferred environment for a "set and forget" workflow.

The recommended tool for this layer is **FolderSync** (or alternatively **Dropsync/Autosync**).[1] Unlike standard Google Drive or Dropbox apps, which are designed for manual interaction, FolderSync operates as a background service that monitors specific local directories.

- **Synchronization Logic:** The user records audio using a high-quality local recorder (e.g., "Easy Voice Recorder" or "Google Recorder"). These apps save to a local path, typically /Music/Recordings/. FolderSync detects the file_created event and initiates an immediate upload to the designated cloud "Ingest" folder.
- **Protocol Selection:** The system should utilize a standard protocol like **WebDAV** or the native APIs of Dropbox/Google Drive. Dropbox is technically superior for this specific use case due to its faster delta-sync capabilities and "LAN Sync" features, which can speed up transfer if the processing computer is on the same network.
- **Metadata Preservation:** Critical to the "arrange files" requirement is the preservation of file creation timestamps. FolderSync maintains the mtime (modified time) attribute during transfer, which serves as the fallback sorting key if the AI fails to extract a narrative date.

3.1.2 iOS Architecture: The Shortcuts Automation

iOS presents a "Walled Garden" challenge where apps cannot freely monitor each other's storage. Background sync is restricted. To achieve the user's goal of using "normal voice recordings" without manual upload, we must leverage **Apple Shortcuts**.[3]

The recommended architecture involves a custom "Dictate Book" Shortcut that bypasses the native Voice Memos app's isolation.

- **Workflow Logic:**
  1. **Trigger:** User taps a home screen icon or uses a Siri command ("Hey Siri, Record Book Entry").
  2. **Capture:** The Shortcut executes the Record Audio action. Crucially, the quality settings must be set to "High" (generating an m4a/AAC file) rather than "Normal" to ensure the STT model captures semantic nuances.
  3. **Metadata Injection:** Before saving, the Shortcut can prompt the user for a "Slug" or "Topic" via a text input or dictation. This string is appended to the filename (e.g.,

2026-01-26_The_Incident_at_The_Docks.m4a).

4. **Transport:** The Shortcut executes Save File, directing the output to a specific iCloud Drive folder or, via API call, to a Dropbox folder. iCloud Drive is the path of least resistance on iOS, as it synchronizes natively to macOS and Windows (via the iCloud for Windows client).

Comparative Analysis of Ingestion Methods:

Feature	Android (FolderSync)	iOS (Shortcuts)	Native Cloud Apps
Automation Level	**High** (Zero-touch background sync)	**Medium** (Requires specific trigger)	**Low** (Manual "Share" action)
Reliability	High (Retries on failure)	Medium (Script execution limits)	High
Metadata Control	Limited to file attributes	High (Custom file naming)	Limited
Setup Complexity	Moderate (One-time config)	High (Visual coding)	Low

For this project, **Android** offers the superior experience for "normal voice recordings" because it allows the user to use their preferred recording hardware/software without changing their behavior. However, the iOS Shortcut method provides better *pre-classification* data by allowing the user to tag the file before upload.

3.2 The Watch Folder Architecture

On the processing side (the server or local PC), a **Watch Folder** acts as the trigger for the entire pipeline. This is managed by a Python script utilizing the watchdog library.[5]

The script must implement a **Debouncing Mechanism**. Large audio files do not appear instantly; they upload in chunks. If the script triggers the moment the file header is created, the transcription will fail on a corrupted/incomplete file.

- **Implementation Detail:** The script should monitor the FileSystemEvent.CREATED event but then enter a polling loop, checking the file size every 5 seconds. Only when the file size remains constant for two consecutive checks (indicating the upload has finished and the file handle is released) does the pipeline proceed to the Transcription Layer.

4. The Transcription Layer: The Neural Ear

Once the audio is ingested, the system must convert the acoustic signal into text. For a book-writing project, the transcription engine must excel at **proper noun recognition** (names of characters), **punctuation** (critical for narrative flow), and **long-form coherence**.

4.1 Model Selection Landscape (2025-2026)

The market is bifurcated into API-based services (Deepgram, Google) and open-weights models (OpenAI Whisper).

4.1.1 OpenAI Whisper V3: The Accuracy Benchmark

Whisper Large V3 stands as the premier choice for this specific application.[7] Unlike traditional phoneme-based ASR (Automatic Speech Recognition) systems, Whisper is trained on 680,000 hours of multilingual web audio using a sequence-to-sequence Transformer architecture.

- **Why it fits:** Whisper is semantically aware. If the narrator says, "He read the read book," a standard model might output "He red the red book." Whisper, understanding the context of "reading," is more likely to output "He read the red book." This semantic awareness is non-negotiable for transcribing fiction or complex narrative, where homophones are context-dependent.
- **The "Turbo" Variant: Whisper Large V3 Turbo** [7] offers a significant optimization. It reduces the decoder layers from 32 to 4, resulting in a model that is 6x faster with negligible accuracy loss (WER increases from ~5.6% to ~5.8%). For a user recording hours of audio, this speed difference is the gap between real-time feedback and overnight processing.

4.1.2 Deepgram Nova-2: The Latency and Diarization Specialist

While Whisper reigns in raw transcription accuracy, **Deepgram Nova-2** is the superior choice if the "incidents" involve dialogue with other people (e.g., interviews).[8]

- **Diarization:** Deepgram's speaker diarization (identifying "Speaker A" vs. "Speaker B") is currently faster and more coherent than the Pyannote-audio pipeline typically used with Whisper.[9]
- **Cost vs. Privacy:** Deepgram is an API service (~$0.0043/min), implying data leaves the user's premise. Whisper can be run locally (offline), offering total privacy for sensitive memoirs.

Recommendation: Given the requirement to "write a book based on... story I narrate," the primary use case is single-speaker dictation. Therefore, **Whisper Large V3 Turbo**, deployed locally via faster-whisper, is the optimal choice. It provides the highest fidelity for narrative

prose without recurring costs.

4.2 Handling Hallucinations and Silence

Transformer-based ASR models like Whisper have a known pathology: **Hallucination during silence**. When fed background static, Whisper often attempts to "predict" the next token, sometimes outputting training data artifacts like "Subtitle by Amara" or "Thanks for watching".[9]

To mitigate this in an automated pipeline, we must integrate a **Voice Activity Detection (VAD)** pre-filter.

- **Technical Implementation:** The Python script should utilize **Silero VAD** or pyannote-audio. This module scans the audio file *before* transcription, creating a timestamp map of speech segments.
- **The Logic:** The system cuts the audio into chunks containing *only* speech, feeds these to Whisper, and then stitches the text back together based on the timestamps. This eliminates 99% of hallucination errors and significantly speeds up processing by ignoring silence.

---

5. The Semantic Analysis Layer: The Cognitive Brain

This section addresses the user's sophisticated requirements: "arrange files automatically according to event" and "always uses a context file to properly arrange and extract the event." This moves beyond transcription into **Knowledge Management**.

5.1 The "Context File" Architecture: The Story Bible

The "Context File" cannot be a unstructured text document. To be useful for "extracting the event" and "arranging files," it must be a **Structured Database** that the AI can query and update. We propose using **YAML** or **JSON** as the storage format because they are both machine-parsable and human-readable.[10]

The Schema Design:

The system maintains a master file, story_bible.json, containing the following objects:

1. **Timeline**: An array of events, each with a narrative_date, recording_id, summary, and status.
2. **Entities**: A dictionary of characters and locations.
3. **Open_Loops**: A list of unresolved plot threads.
4. **Style_Guide**: Specific instructions on voice, tone, and forbidden words.

Example JSON Structure:

JSON

```
{
 "project_title": "The Glass Horizon",
 "timeline": [
  {
   "event_id": "evt_001",
   "narrative_date": "2045-05-12",
   "description": "Protagonist finds the encrypted drive.",
   "source_files": ["2026-01-26_recording.mp3"]
  }
 ],
 "characters": {
  "Kael": {
   "description": "Cynical hacker, prosthetic left arm.",
   "status": "Alive"
  }
 },
 "style_profile": {
  "narrative_voice": "First Person, Noir",
  "sentence_structure": "Short, punchy sentences. Heavy use of sensory metaphor."
 }
}
```

5.2 The Event Extraction Agent

Once the transcript is generated, a specific AI agent (The "Analyst") is triggered. Its sole job is **Structured Information Extraction**.[12]

- **Model: GPT-4o** or **Gemini 1.5 Flash**. These models excel at instruction following and JSON formatting.
- **The Prompt:** The system feeds the raw transcript and the *current* story_bible.json to the Analyst.
  - *Instruction:* "Analyze the transcript. Identify the event described. Determine where this event fits in the timeline based on context clues (e.g., 'Before I met Kael...'). Extract any new character details. Return the update as a JSON object."

- **The Logic:** The Python script uses a library like **Pydantic** to validate the output. If the AI returns valid JSON, the script programmatically updates story_bible.json. This satisfies the requirement to "always use a context file" to extract the event.

5.3 The "Librarian" Model: Context Caching vs. RAG

The user wants the AI to write a book. A novel (80,000+ words) exceeds the standard "attention" span of many models if simply pasted into a prompt.

- **Why RAG (Retrieval Augmented Generation) Fails:** RAG retrieves snippets based on keyword similarity.[14] If the user asks to "Write the scene where Kael dies," RAG works. But if the user says "Continue the story," RAG struggles to retrieve the *thematic* arc or the *pacing* established 50 pages ago. RAG fractures the narrative coherence.
- **The Solution: Long Context Caching:** In 2025, the dominant architecture for long-form content is **Context Caching**, pioneered by Google (Gemini) and Anthropic (Claude).[15]
    - **Mechanism:** Instead of re-sending the 50,000-word draft with every request (which is slow and expensive), we "cache" the entire book and the Story Bible in the model's memory.
    - **Gemini 1.5 Pro:** Supports a **2 Million Token** context window with caching. This allows the model to "hold" the entire trilogy in its head while writing the next chapter.
    - **Cost Efficiency:** Cached input tokens are significantly cheaper (~$0.31 per 1M tokens vs $1.25 for standard input).[18] This makes it economically feasible to check the *entire* history of the book for every single new scene generation, ensuring perfect continuity.

Recommendation: The stack uses **Gemini 1.5 Pro** as the "Librarian" to maintain the state and consistency of the massive story_bible.json, as its massive window allows it to cross-reference hundreds of separate events without information loss.

---

6. The Stylometric Generation Layer: The Ghostwriter

The requirement "format the text how I write and narrate story" demands **Style Transfer**. Generic AI models have a specific "AI accent" (overuse of words like "delve," "tapestry," "testament"). To sound like the user, we need a specialized approach.

6.1 Model Selection for Prose

While Gemini excels at context, **Anthropic's Claude 3.5 Sonnet** is currently the industry leader for creative prose generation.[19]

- **Stylistic Adherence:** Benchmarks show Claude 3.5 Sonnet adheres more strictly to

negative constraints (e.g., "Do not use flowery adjectives") than GPT-4o, which often reverts to its RLHF (Reinforcement Learning from Human Feedback) training bias of being "helpful and polite" rather than "gritty and authentic."
- **Steerability:** Claude's "System Prompt" adherence allows for creating a detailed "Persona" that mimics the user.

6.2 Capturing the User's Voice: Few-Shot Prompting

We do not necessarily need to fine-tune a model (which is expensive and static). Instead, we use **Dynamic Few-Shot Prompting**.

1. **Analysis Phase:** The user provides 3-5 samples of their previous writing.
2. **Stylometric Extraction:** An LLM analyzes these samples to generate a style_profile (stored in the Story Bible). It explicitly defines the user's sentence length variance, vocabulary complexity, and preferred metaphors.
3. **Generation Phase:** When writing a new scene, the Python script injects this style_profile and 2-3 snippets of the user's *actual writing* into the prompt as examples.
    - *Prompt:* "Write the following scene based on the transcript. Adopt the style of the following examples:. Adhere to the Style Profile: [Insert Profile]."

This ensures the output feels authentic to the user, satisfying the "format how I write" requirement.

---

7. The File Organization & Compilation Layer

The final output must be a book, not a folder of loose text files.

7.1 Automated Sorting Logic

The "arrange files automatically according to event" requirement is handled by the **Event Extraction Agent** (Phase 5.2).

- **Renaming Protocol:** Once the agent identifies the event's place in the timeline, the Python script renames the Markdown file.
    - *From:* 2026-01-26_Recording.md
    - *To:* Drafts/Sequence_045_The_Incident_at_Docks.md
- **Frontmatter Injection: The script injects YAML Frontmatter at the top of every text file.**[10]

sequence_id: 45 narrative_date: 2045-05-12 characters: source_audio: 2026-01-26_Recording.mp3
The Incident at the Docks
The water was black...
This metadata allows the system to re-sort the files programmatically at any time, regardless of when they were recorded.

7.2 The Compiler: Pandoc

To produce the final artifact, we utilize **Pandoc**, the "Swiss Army Knife" of document conversion.[21]

- **The Compiler Script:** A script (compile.py) scans the Drafts/ folder.
- **Ordering:** It reads the narrative_date or sequence_id from the YAML frontmatter of every file to determine the correct order of chapters.
- **Stitching:** It merges them into a single Master_Manuscript.md.
- **Output:**
  - **EPUB:** For reading on Kindle/Tablets (using a custom CSS style sheet).
  - **PDF:** For print layout (using LaTeX templates for professional typesetting).
  - **DOCX:** For editing.

This satisfies the need for the book to be "arranged" and formatted automatically.

---

8. Automation & Orchestration: The Python Logic

The glue holding this stack together is a custom Python application. This section outlines the logic flow and library requirements.

8.1 The "Watcher" Script

We utilize the watchdog Python library to monitor the ingestion folder.

Python

```python
import time
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class AudioHandler(FileSystemEventHandler):
    def on_created(self, event):
        if event.is_directory: return
        filename = event.src_path
        if filename.endswith(('.mp3', '.m4a', '.wav')):
            print(f"New audio detected: {filename}")
            process_pipeline(filename)

def process_pipeline(filepath):
    # Step 1: Debounce (Wait for upload to finish)
    wait_for_file_stability(filepath)

    # Step 2: Transcribe
    transcript = run_whisper(filepath)

    # Step 3: Extract Event & Update Context
    context = load_json("story_bible.json")
    event_data = run_analyst_agent(transcript, context)
    update_story_bible(event_data)

    # Step 4: Write Prose
    prose = run_writer_agent(transcript, context, event_data)

    # Step 5: Save & Organize
    save_to_drafts(prose, event_data)
```

8.2 Error Handling and Resilience

- **Network Failure:** If the API (Claude/Gemini) fails, the script must queue the job and retry. It should not crash or lose the audio file.
- **Backup:** The script should treat the Input folder as read-only or move processed files to an Archive folder, never delete them. This ensures the raw voice data—the single source of truth—is never lost.

---

9. Cost & Feasibility Analysis (2025/2026 Pricing)

Building this stack involves both fixed and variable costs.

Table 2: Estimated Operational Costs (Per 100,000 words generated)

Component	Service	Cost Basis	Est. Monthly Cost
Storage	Google Drive / Dropbox	2TB Plan	~$10.00
Transcription	Whisper API	$0.006/min	~$6.00 (for 1000 mins)
Context Memory	Gemini 1.5 Pro Caching	$4.50/1M tokens/mo	~$9.00 (for 2M cache)
Generation	Claude 3.5 Sonnet	$3.00/1M Input	~$15.00
Orchestration	Local Python Script	Electricity	Negligible
Total			**~$40.00 / month**

Note on Savings: Running Whisper locally (if the user has a GPU) saves the transcription cost entirely. However, the Context Caching cost is unavoidable for a "Story Bible" feature of this magnitude unless running a high-end local LLM (like Llama 3 70B), which requires expensive hardware ($1,500+ GPU).

---

10. Conclusion and Recommended Tech Stack

To satisfy the user's request for an automated, context-aware, style-matching book writing system, the following "Best Tech Stack" is recommended:

1. **Ingestion: FolderSync** (Android) or **Shortcuts** (iOS) syncing to **Dropbox**.
    - *Reason:* Frictionless, background automation.
2. **Transcription: Faster-Whisper (Large-V3-Turbo)** running locally via Python.
    - *Reason:* Best-in-class accuracy, semantic awareness, zero cost.
3. **Context Database: JSON** file (story_bible.json) managed via **Pydantic** schema validation.
    - *Reason:* Structured, machine-parsable memory of the story.
4. **Context Intelligence: Google Gemini 1.5 Pro** with **Context Caching**.
    - *Reason:* The "Librarian" that holds the entire book in memory to ensure continuity.
5. **Prose Generation: Anthropic Claude 3.5 Sonnet** (API).
    - *Reason:* The "Ghostwriter" with the best prose quality and style adherence.
6. **Orchestration: Python** using watchdog for file monitoring and asyncio for API calls.
7. **Compilation: Pandoc** with **LaTeX** templates.
    - *Reason:* Professional-grade formatting of the final manuscript.

This architecture provides a complete end-to-end pipeline: from the spoken word on a walk in the park to a formatted book chapter on the desktop, all processed in the background with strict narrative consistency.

Works cited

1. Dropsync: Autosync for Dropbox - Apps on Google Play, accessed January 26, 2026, https://play.google.com/store/apps/details?id=com.ttxapps.dropsync
2. Autosync for Google Drive - Apps on Google Play, accessed January 26, 2026, https://play.google.com/store/apps/details?id=com.ttxapps.drivesync
3. Enable iCloud Drive sync using Shortcuts' personal automations, accessed January 26, 2026, https://forum.literatureandlatte.com/t/enable-icloud-drive-sync-using-shortcuts-personal-automations/128759
4. Voice record daily thoughts, redact with GPT4, and save to Apple ..., accessed January 26, 2026, https://jakobs.dev/voice-record-daily-notes-redact-gpt4-apple-shortcuts/
5. Automatically Transcribing Files in Watch Folders, accessed January 26, 2026, https://macwhisper.helpscoutdocs.com/article/35-automatically-transcribing-files-in-watch-folders
6. How do I run Whisper on an entire directory? - Stack Overflow, accessed January 26, 2026, https://stackoverflow.com/questions/74844226/how-do-i-run-whisper-on-an-entire-directory
7. Best open source speech-to-text (STT) model in 2026 ... - Northflank, accessed January 26, 2026, https://northflank.com/blog/best-open-source-speech-to-text-stt-model-in-202

5-benchmarks
8. Best STT Providers 2025 | Deepgram vs Google vs Azure - Edesy, accessed January 26, 2026, https://edesy.in/ai-voice-assistant/tools/stt-comparison
9. DIY Transcription app: How to set up OpenAI's Whisper on your laptop, accessed January 26, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1ew4gzf/diy_transcription_app_how_to_set_up_openais/
10. YAML Frontmatter In Markdown Files: A Dprint Guide, accessed January 26, 2026, https://www.falldrive.dsausa.org/avoid/yaml-frontmatter-in-markdown-files
11. Parsing YAML out of a Markdown file - python - Stack Overflow, accessed January 26, 2026, https://stackoverflow.com/questions/57168926/parsing-yaml-out-of-a-markdown-file
12. Introducing Structured Outputs for Batch and Agent Workflows, accessed January 26, 2026, https://www.databricks.com/blog/introducing-structured-outputs-batch-and-agent-workflows
13. Structured outputs | LLM Inference Handbook - BentoML, accessed January 26, 2026, https://bentoml.com/llm/getting-started/tool-integration/structured-outputs
14. Retrieval Augmented Generation or Long-Context LLMs? A ..., accessed January 26, 2026, https://aclanthology.org/2024.emnlp-industry.66.pdf
15. Gemini CLI Pricing Guide - Complete Cost Analysis 2025, accessed January 26, 2026, https://geminicli.org/pricing
16. LLM API Pricing Comparison (2025): OpenAI, Gemini, Claude, accessed January 26, 2026, https://intuitionlabs.ai/pdfs/llm-api-pricing-comparison-2025-openai-gemini-claude.pdf
17. Prompt Caching: Cost & Performance Analysis Across Providers, accessed January 26, 2026, https://artificialanalysis.ai/models/caching
18. Peculiar Pricing of Context Caching and Potential Plans for Prefix ..., accessed January 26, 2026, https://discuss.google.dev/t/peculiar-pricing-of-context-caching-and-potential-plans-for-prefix-caching-support/194137
19. The Best LLM for Content Creation... | by Harshit Tyagi - Medium, accessed January 26, 2026, https://dswharshit.medium.com/the-best-llm-for-content-creation-06dd1ee5d7b9
20. 11 Best LLM Models in 2025: Top Picks and Comparisons - LinkGraph, accessed January 26, 2026, https://www.linkgraph.com/blog/best-llm-model/
21. How toUse Pandoc to convert a Markdown to a PDF and ePUB, accessed January 26, 2026, https://sergiodxa.com/tutorials/use-pandoc-to-convert-a-markdown-to-a-pdf-and-epub
22. Pandoc User's Guide, accessed January 26, 2026,

https://pandoc.org/MANUAL.html